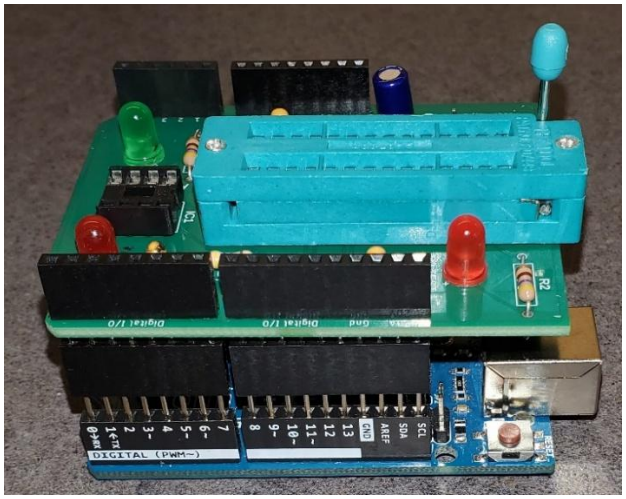
Bare Bones Programming Shield User Manual

By: Jim Schrempp and Bob Glicksman; v1, 3/18/2026

NOTICE: Use of this document is subject to the terms of use described in the document "Terms_of_Use_License_and_Disclaimer" that is included in this release package. This document can be found at:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/blob/main/Terms_of_Use_License_and_Disclaimer.pdf



© 2026 Team Practical Projects, Bob Glicksman, and Jim Schrempp. All rights reserved.

TABLE OF CONTENTS.

TABLE OF CONTENTS. 1

OVERVIEW. 2

“BARE BONES” MICROCONTROLLERS. 3

USING THE PROGRAMMER. 4

SET UP THE ARDUINO IDE FOR “BARE BONES” CHIP PROGRAMMING. 6

Setup for ATMega328(P). 7

Setup for ATtiny85. 9

ONE-TIME CONFIGURATION OF “FACTORY FRESH” CHIPS. 12

Configure ATmega328(P). 12

Configure ATtiny85. 13

COMPILING AND UPLOADING YOUR PROGRAM TO A “BARE BONES” CHIP. 15

Compile and Upload to Atmega328(P). 15

Compile and Upload to ATtiny85. 17

APPENDIX A. ATMEGA328(P) PIN MAPPING. 18

APPENDIX B. ATTINY85 PIN MAPPING. 19

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

OVERVIEW.

This document describes how to use the ***Bare Bones Programming Shield*** (BBP) from Team Practical Projects. The BBP allows the user to utilize the Arduino IDE to compile and upload programs to the following "bare bones" microcontroller chips:

- **Atmega328(P)**. This is the same microcontroller as used on the Arduino Uno R3 and others. Digikey part: https://www.digikey.com/en/products/detail/microchip-technology/atmega328p-pu/1914589?_gl=1*nhrfbt*_up*MQ..*_gs*MQ..&gclid=CjwKCAjwjtTNBhB0EiwAuswYhnvZ8xKeAaPThXu-ZLJ6agSblxGYiEhnb03qzDWQN2VRzf35oOe8fxoCE_YQAvD_BwE&gclsrc=aw.ds&gbraid=0AAAAADrbLlhgVJPxqyqgm26uFk-9uBpPM

- **ATtiny85**. This is a small (8 pin) microcontroller that is largely compatible with the Atmega328 microcontroller but contains fewer I/O pins and less memory. It is suitable for smaller, lower cost projects. Digikey part: https://www.digikey.com/en/products/detail/microchip-technology/ATTINY85-20PU/735469

The BBP is an open source project and you can build and configure your own version. Details are in the document:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/blob/main/Documentation/Bare_Bones_Programmer_Assembly_and_Theory_Manual.pdf

This document assumes that you have constructed your own BBP and configured both the programmer and the Arduino IDE as detailed in the aforementioned manual. The "programmer" used for this project consists of an Arduino Uno (R3 or earlier version) and a custom-built Arduino shield board. Electronic CAD files for the custom shield board (Fusion 360 electronics) are included in the open source project repository:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/tree/main/Hardware

- Fusion 360 .fsch and .fbrd files.
- Bare_Bones_Programming_Shield v17_2026-02-19.zip contains manufacturing ("Gerber") files in a zip archive that is suitable for uploading to an on-line PCB manufacturing site such as JLCPCB.com, PCBWay.com, OSHPark.com, etc[1].

[1] Our boards were manufactured by JLCPCB.com, so we can verify that the ZIP archived files work correctly with this site.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

The complete project repository, including manufacturing files and documentation can be found at:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield

"BARE BONES" MICROCONTROLLERS.

A "bare bones" microcontroller, in the context of this project, means the bare microcontroller chip, as purchased right from the distributor ("factory fresh") and used in your microcontroller project as-is, meaning without any external components[2] to support the microcontroller's operation. When these microcontrollers are configured for "bare bones" operation, they will run using their internal clock at 8 MHz. This is the maximum clock frequency supported for internal clocking which also allows these microcontrollers to run with supply voltages as low as 2.7 volts (up to 5.5 volts).

The advantages of using a "bare bones" microcontroller chip in lieu of a microcontroller board or module are:

- Lower cost.
- Smaller footprint.
- Wider range of power supply voltages (e.g. when powering from batteries)
- Ability to power down the microcontroller to extremely low power draw while retaining the program and data state.

Disadvantages of using a "bare bones" microcontroller chip in lieu of a board or module are:

- Possibly lower clock speed (8 MHz, vs 16 MHz for Arduino Uno or Nano).
- Need to place the chip in a dedicated "programmer" device in order to upload code to it.
- Need an external device ("FTDI" board) in order to utilize the serial monitor with your program.

When using a "bare bones" microcontroller chip in your project, you must map the Arduino software "pins" to the physical hardware pins on the chip that you are using. That is, you write your software using the Arduino software pins (e.g. D3, A1, RxD and TxD, ….) but you wire up the corresponding hardware pins (the pins on the chip itself). See the appendices at the end of this document for the mappings between Arduino software pins and chip hardware pins.

When using a factory fresh microcontroller chip, you must first configure the chip for "bare bones" operation (internal 8 MHz clock) by setting fuses inside the chip. This is a one-time

[2] Power supply decoupling capacitors are still necessary, of course, but external crystals and other components are not.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

operation. Once the chip is configured, you need not configure it again[3]. Therefore, you can skip this step and go directly to “Compiling and Uploading …” after the chip is initially configured.

USING THE PROGRAMMER.

Figure 1 is a photo of the programmer for this project. It consists of a custom designed Arduino shield board that is stacked on top of an Arduino Uno. The Uno must be version R3 or earlier, i.e. must use the 8 bit ATmega328P microcontroller (not the Uno R4 or Q, which use 3.3 volt, 32 bit microcontrollers).

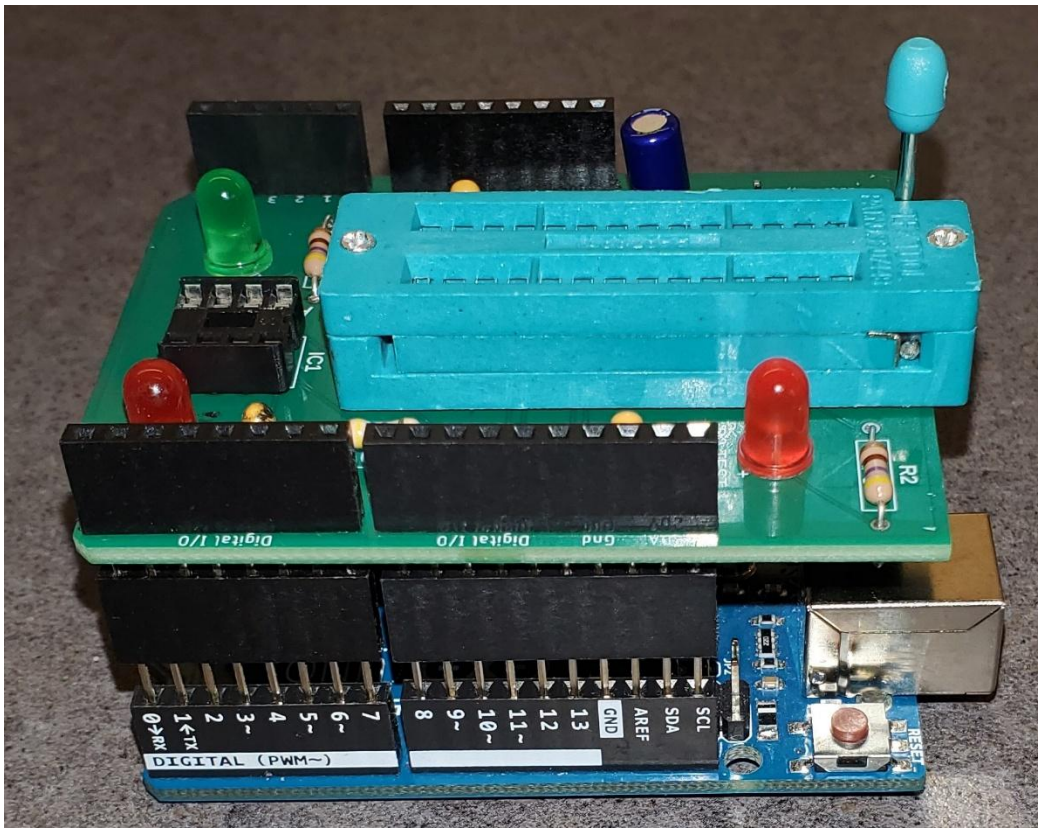


Figure 1. Photo of the Programmer.

The photo in figure 1 shows a zero insertion force (ZIF) socket for programming the 28 pin ATmega328(P) microcontroller chips. An ordinary 28 pin DIP socket may be used. If so, care must be taken when inserting the chip to ensure that all pins are correctly inserted into the socket. The 8 pin ATtiny85 is less of a problem since it only has 8 pins and not 28 pins. An ordinary 8 pin DIP socket is shown in figure 1.

[3] It is perfectly OK to configure the chip multiple times, i.e. if you forgot whether the chip was previously configured.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

Figure 2, below, is an annotated view of the shield board. The shield board is where you plug in your bare bones microcontroller chip in order to upload programs to it using the Arduino IDE,

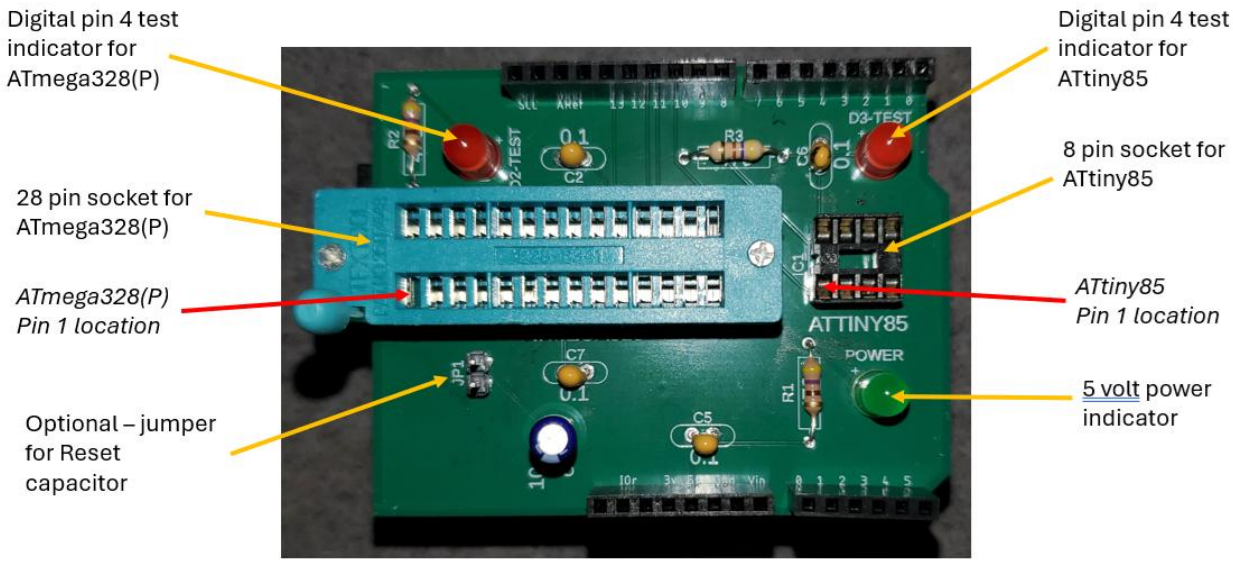


Figure 2. Annotated View of the Programmer Shield.

The shield board contains a 28 pin socket that accepts the ATmega328(P) chip for programming and an 8 pin socket that accepts the ATtiny85 chip for programming. Note that pin 1 of both sockets is to the left, as shown in figure 2. *Care must be taken to plug the microcontroller chip into the correct socket in the correct orientation!*

The shield board receives 5 volt power from the underlying Arduino Uno board. The Arduino Uno, in turn, is usually powered by plugging its USB connector into a USB port on the computer that hosts the Arduino IDE. *NOTE: it is strongly recommended that the Uno be unpowered (unplugged) when inserting and removing microcontroller chips on the shield board*.

In order to upload programs to a “bare bones” microcontroller chip:

- Unplug the Uno’s USB cord so that the BBP is unpowered.
- *CAREFULLY* insert either an ATmega328(P) chip into the 28 pin socket or an ATtiny85 chip into the 8 pin socket. *Double check that chip pin one is correctly aligned in the socket and that all chip pins are correctly inserted into their respective sockets*. If a ZIF socket is used, move the handle to the locked position after the chip is inserted into the socket.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

- Plug the Uno's USB port into a USB port on the computer that you will use to compile and upload your program to the chip. The green LED (5 volt power indicator) should light up, indicating that the Uno and shield board have power.
- Follow thew instructions below to configure the chip as a "bare bones" microcontroller (one time only) and to compile and upload your program to the chip.
- When programming is complete, unplug the USB cable from the Uno to remove power to the shield board.
- *CAREFULLY* remove your now-programmed microcontroller from its socket. If a ZIF socket is used, move the handle up to the unlocked position in order to remove the chip. You can now place the chip into your project.

Additional components are shown on the shield board in figure 2:

- Two red LEDs. One of these LEDs is connected to pin 6 of the 28 pin socket and the other red LED is connected to pin 3 of the 8 pin socket. Both of these pins map to Arduino digital pin 4 in Arduino software. A "blink" program that blinks an LED on Arduino digital pin 4 may be compiled and uploaded to one of these chips while it is still in the programmer socket. The appropriate red LED will blink, ensuring that all steps in the programming process have been done correctly.
- A jumper. If a jumper is inserted between these two pins, a 10 uF capacitor is connected between the !RESET pin on the Uno and ground. *This configuration is not necessary for this project and a jumper should **NOT** be inserted across these pins!*

SET UP THE ARDUINO IDE FOR "BARE BONES" CHIP PROGRAMMING.

The Arduino IDE must be set up to program the "bare bones" chip that you will be using. The "off the shelf" Arduino IDE does not provide support for these "bare bones" chips. Support files must be added to the IDE. Complete instructions for doing this are contained in the document:

https://github.com/TeamPracticalProjects/Bare_Bones_Programming_Shield/blob/main/Documentation/Bare_Bones_Programmer_Assembly_and_Theory_Manual.pdf

The Arduino Uno that is part of the programmer (under the shield board) must be running a special program called "Arduino ISP". Instructions for uploading this program to the Uno are contained in the document above.

It is helpful, but not essential, to change the Preferences in the Arduino IDE, per figure 3, below.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

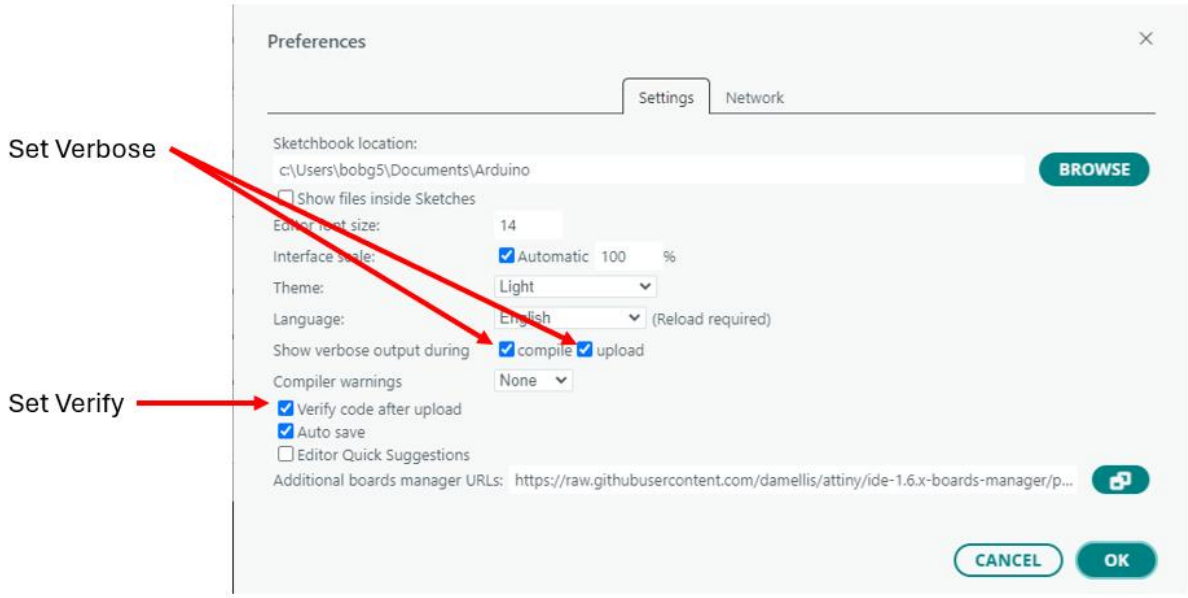


Figure 3. Arduino ide Preferences.

We suggest setting verbose output for both compile and upload, so that it is easy to see what is happening in the event of an error when trying to compile and upload programs to a bare bones microcontroller chip. We also suggest setting “Verify code after upload” to ensure that the compiled program was successfully uploaded to the bare bones microcontroller chip.

At this point, it is assumed that that all steps in the aforementioned document have been followed and the BBP is ready to program ATtmega328(P) and ATtiny85 microcontroller chips. The Arduino IDE should be opened on the host computer and must be configured for the type of chip being programmed – see the proper subsection, below.

Setup for ATMega328(P).

MAKE SURE THAT YOU HAVE YOUR BARE BONES CHIP PROPERLY INSERTED INTO THE SHIELD BOARD AND THAT THE GREEN POWER LED IS LIT.

Open the “Tools” menu on the Arduino IDE and go to “Port:” – see figure 4, below. Make sure that the proper COM port (Windows) or tty port (Linux or Mac) is selected. This must be the port that the Uno of the BBP is connected to. Note that the port on your computer may be different from that shown in figure 4, below.

Next, go to “Board:” on the “Tools menu. Look down the drop-down list for “ATmega328 on a breadboard (8 MHz internal clock)”, as shown in figure 4 below. Select “ATmega328 on a breadboard (8 MHz internal clock)” as the board to be programmed.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

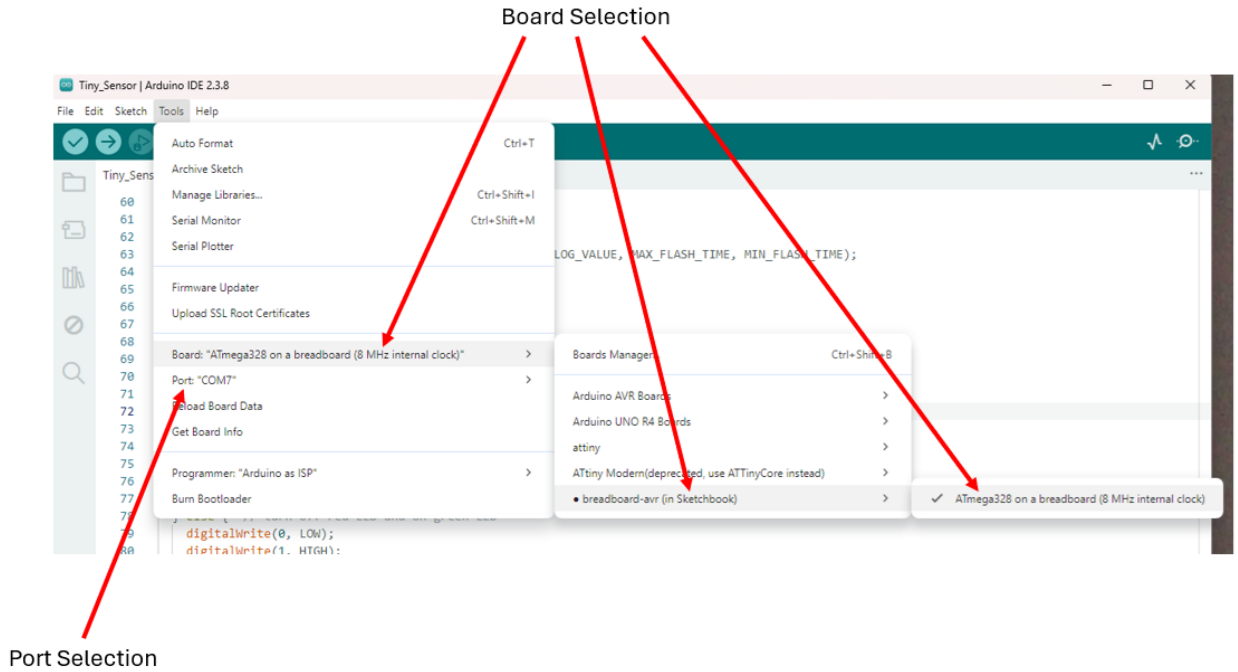


Figure 4. Select the ATmega328 "board" configuration.

Next, go to "Tools" again and change the "Programmer" setting to "Arduino as ISP"; see figure 5, below. *NOTE: take care to select "Arduino as ISP" and not "ArduinoISP".*

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

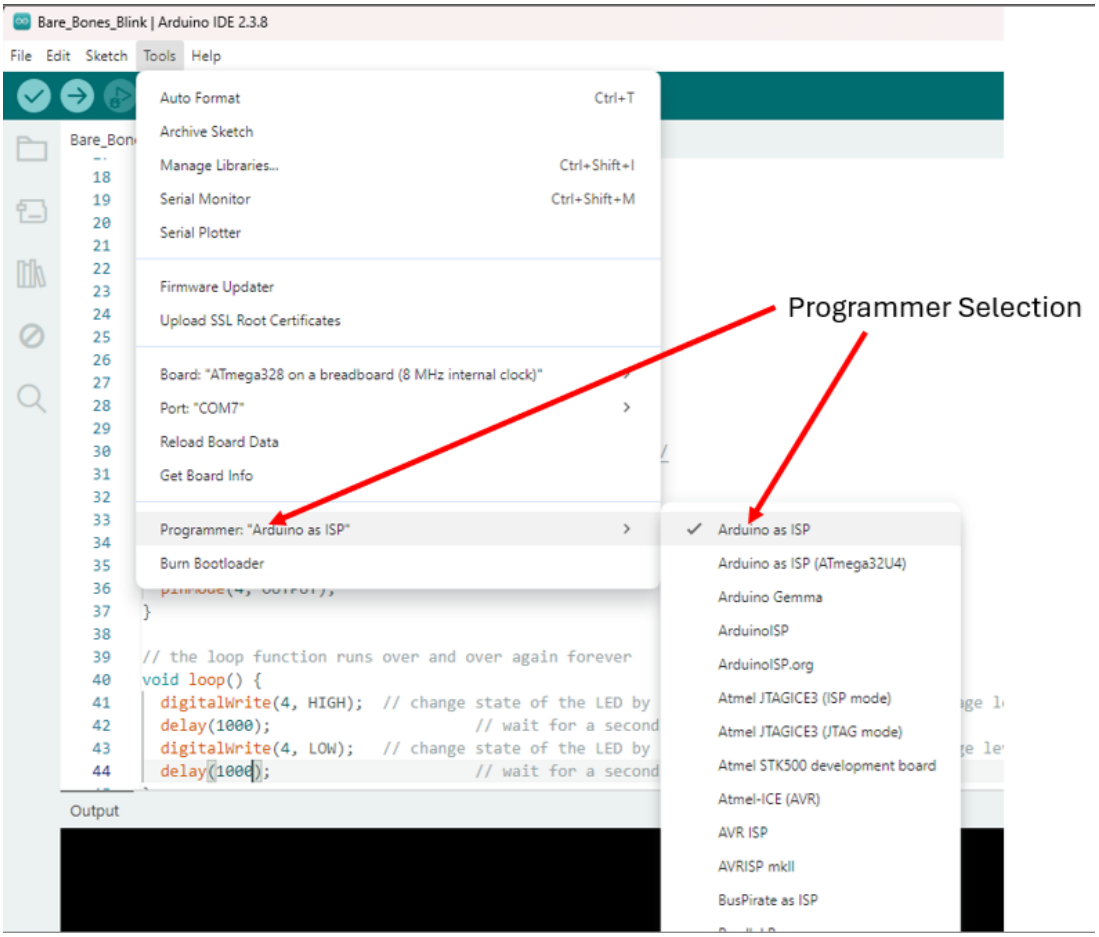


Figure 5. Selecting "Arduino as ISP" as the programmer.

Before proceeding, verify all of your IDE "Tools" settings, as in figure 5.

Setup for ATtiny85.

The "Tools" dropdown in the Arduino IDE must be set up to program an ATtiny85, similarly to configuration described above for the Atmega329(P).

- Open the "Tools" dropdown and look at the "Port:" setting. Make sure that the COM or tty port selected is the port that the Uno that is part of the BBP is selected (not necessarily the port shown in figure 6 below).
- In the IDE, go to "Tools > Board:" and select the "attiny > ATtiny25/45/85".

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

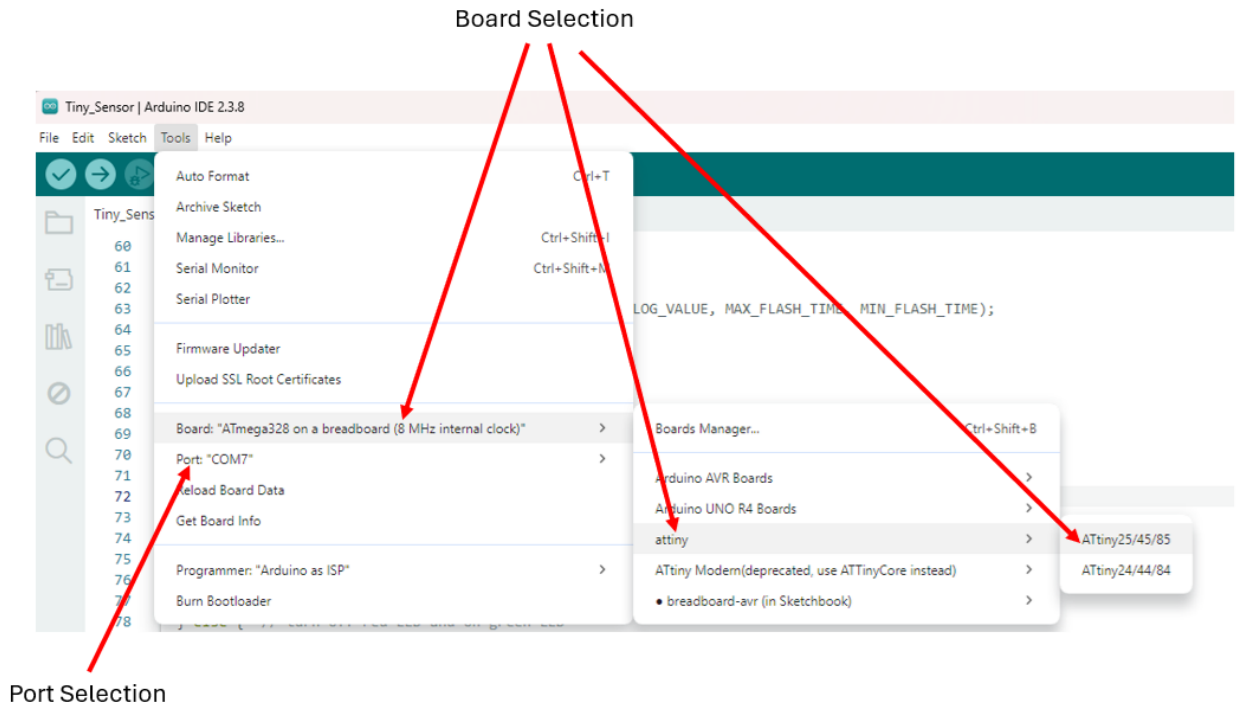


Figure 6. Select the ATtiny "board" configuration.

Note that the selection is for a family of 8 pin microcontrollers – ATtiny 25, 45 and 85. The 25 and 45 chips are the same as the 85 chips, but they have less memory. We suggest that hobbyists always use the ATtiny85, since the cost differential for the less powerful 25 and 45 is negligible.

Next, go to "Tools > Clock:" and select "Internal 8 MHz", as shown in figure 7, below. Then, go to Tools > Processor and select "ATtiny85", as shown in figure 8, below. Lastly, make sure that the "Programmer:" is set to "Arduino as ISP", as shown in figure 8, below. *NOTE: take care to select "Arduino as ISP" and not "ArduinoISP".*

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

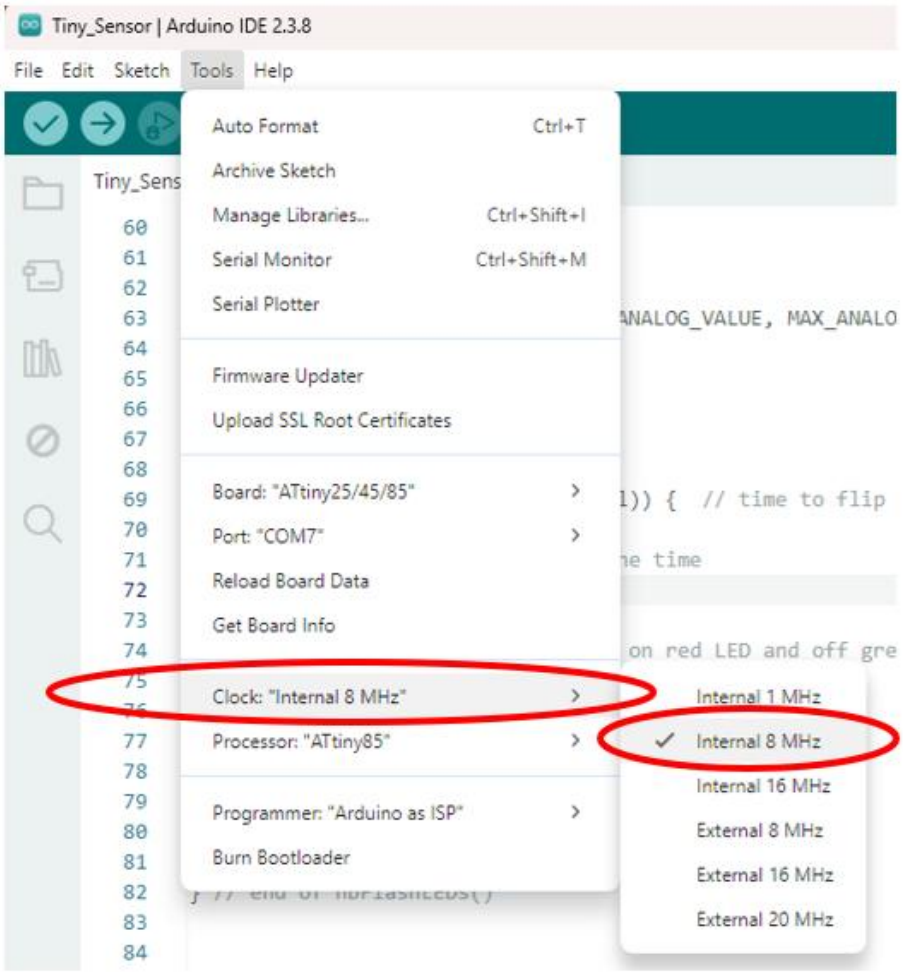


Figure 7. Select the "Clock:" setting for ATtiny 25/45/85.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

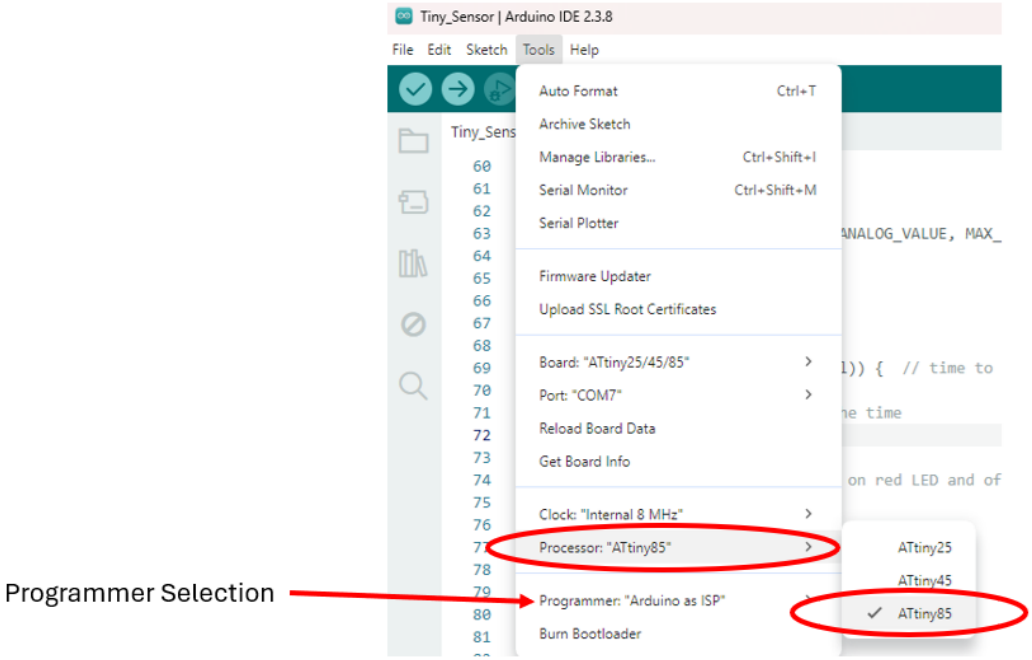


Figure 8. Selecting the ATtiny85 Processor, and "Arduino as ISP" as the Programmer.

Double check all of your "Tools" settings as per figure 8.

ONE-TIME CONFIGURATION OF "FACTORY FRESH" CHIPS.

You must next flash the Arduino bootloader to the bare bones chip. This step is necessary because it sets fuses and locks on the bare bones chip that are needed for the chip to operate properly. This step will also flash the bootloader program to your bare bones chip, but the bootloader is not required and may be overwritten in the subsequent steps of this procedure. *This step is only required once, for a "factory fresh" chip. It need not be repeated once the chip is configured. However, repeating this step will not harm the chip in any way.*

Configure ATmega328(P).

Refer to figure 9, below. Double check the "Tools" settings per this figure. Next, click on "Burn Bootloader" at the bottom of the Tools menu, as indicated in figure 9.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

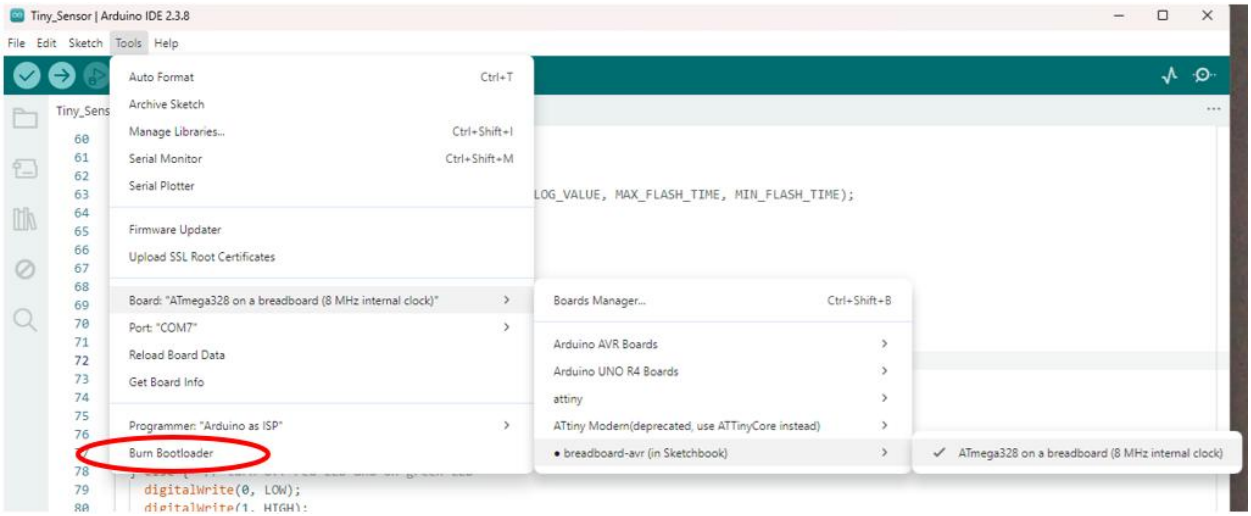


Figure 9. ATmega328 "Burn Bootloader".

You will see status messages in the console window of the IDE as this process proceeds. Double check that the process has finished successfully (no errors indicated) before proceeding.

If you get a signature error in the IDE status window, then you are probably configured for wrong chip. Note that the ATmega329P has a different signature than the non-P version of this chip. The AT series of microcontrollers have a unique signature for each type and version of chip and the IDE will check to make sure that it is programming the correct chip version before proceeding.

Configure ATtiny85.

Refer to figure 10, below. Double check the "Tools" settings per this figure. Next, click on "Burn Bootloader" at the bottom of the Tools menu, as indicated in figure 10.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

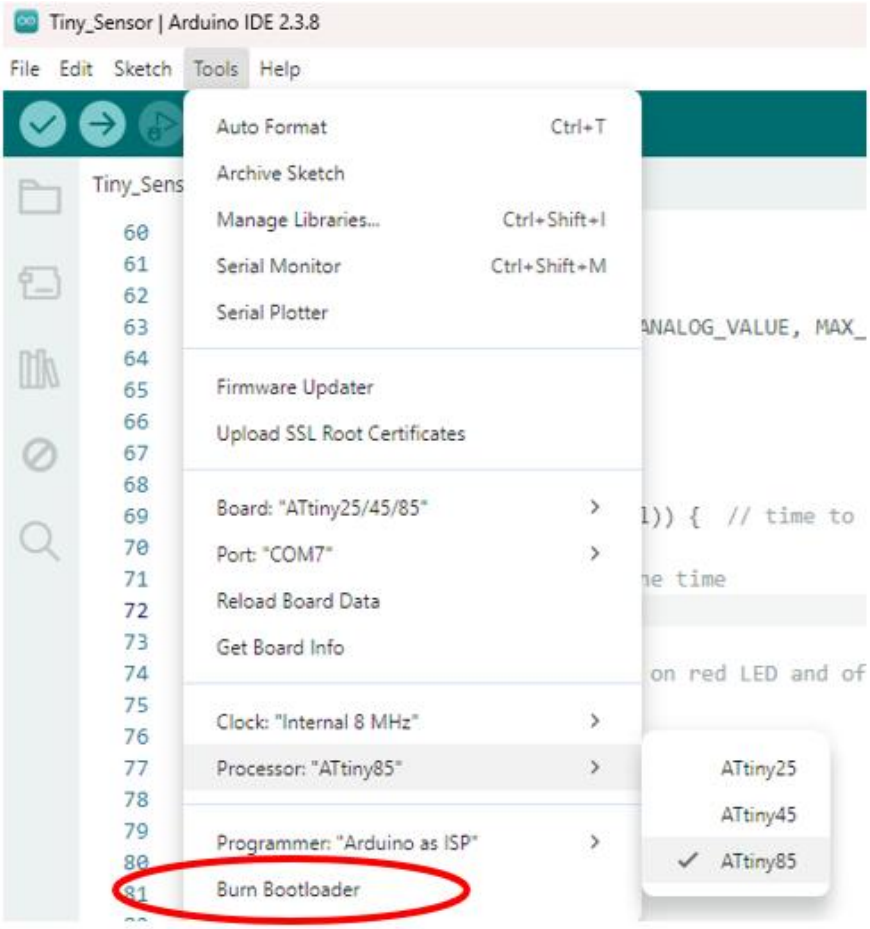


Figure 10. ATtiny85 "Burn Bootloader".

You will see status messages in the console window of the IDE as this process proceeds. Double check that the process has finished successfully (no errors indicated) before proceeding.

If you get a signature error in the IDE status window, then you are probably configured for wrong chip. The AT series of microcontrollers have a unique signature for each type and version of chip and the IDE will check to make sure that it is programming the correct chip version before proceeding.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

COMPILING AND UPLOADING YOUR PROGRAM TO A “BARE BONES” CHIP.

Compile and Upload to Atmega328(P).

You should have the proper settings in the “Tools” menu, but you should verify this, as in figure 9, above. Specifically, double-check the following:

- “Board: ATmega328 on a breadboard (8 MHz internal clock)”
- “Port:” The COM or tty port that your Uno is connected to.
- “Programmer:” “Arduino as ISP”.

You are now ready to open or create a new program of your own that you want to compile and upload to the bare bones chip. Proceed as you would for any program that you import, select, or create on the Arduino IDE. When your program is ready, you can compile and verify (but NOT upload) your code. You do this by clicking on the CHECK icon at the top left of the IDE (NOT the right arrow icon!); see figure 11, below:

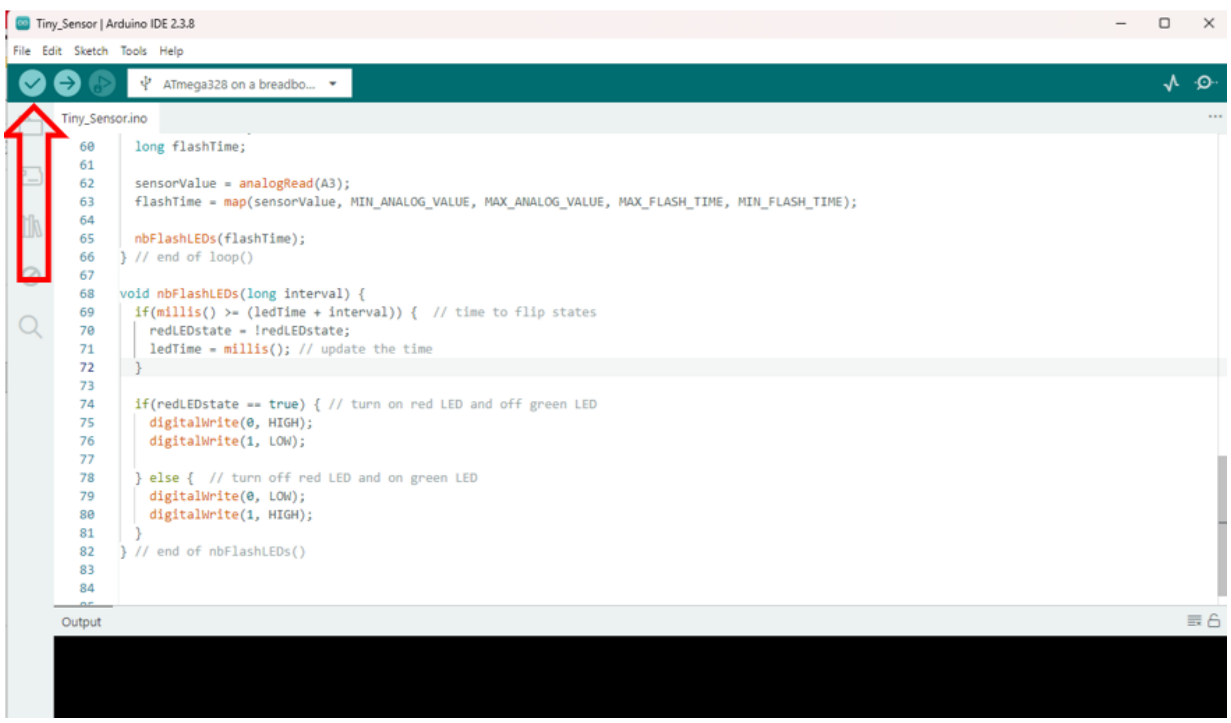


Figure 11. Compile only (Verify).

Check the status messages in the console window of the IDE to make sure that your code has compiled and verified correctly!

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

In order to upload your program to the bare bones chip using the BBP, you must open the "Sketch" menu on the Arduino IDE and select "Upload Using Programmer", see figure 12, below. *NOTE: You MUST use "Sketch > Upload Using programmer" and not click on the right arrow on the IDE toolbar!*

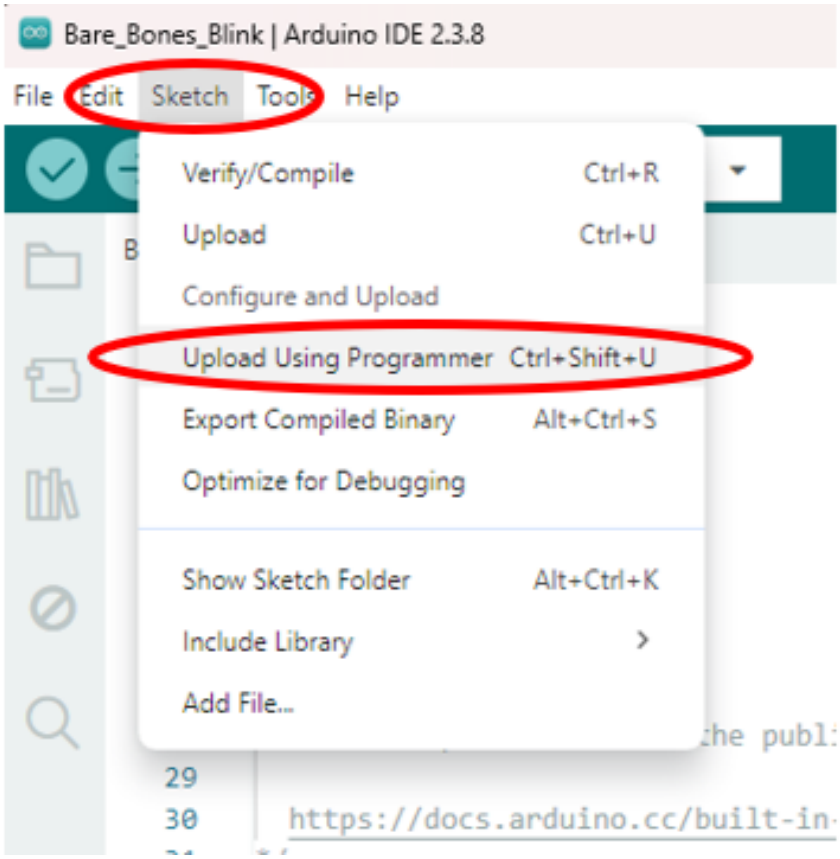


Figure 12. Compiling and Uploading your program to a "bare bones" chip.

You will see status information (in white and red text) in the IDE console window as this process proceeds. The process will automatically:

- Compile your program (again).
- Upload the compiled program to your "bare bones" microcontroller chip.
- Verify that the program has been correctly stored in the chip's program memory.

At this point the Atmega328(P) will be executing code while in the programmer. If you have flashed a blink program (for an LED on Arduino digital pin 4) to the chip, the red LED by the chip will blink at the correct rate, verifying that everything has been done correctly. You can now create/load the program for your project and select "Sketch > Upload Using Programmer" again to compile and upload the new program.

© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

Compile and Upload to ATtiny85.

You should have the proper settings in the “Tools” menu, but you should verify this, as in figure 10, above. Specifically, double-check the following:

- “Port:” the COM or tty port selected is the port that the Uno that is part of the BBP is connected to (not necessarily the port shown in figure 10).
- “Board:” ATtiny25/45/85.
- “Processor:” ATtiny85.
- “Clock:” Internal 8 MHz.
- “Programmer:” Arduino as ISP.

You are now ready to open or create a new program of your own that you want to upload to the bare bones chip. Proceed as you would for any program that you import, select, or create on the Arduino IDE. When your program is ready, you can compile and verify (but NOT upload) your code. You do this by clicking on the CHECK icon at the top left of the IDE (NOT the right arrow icon!), as shown in figure 11, above.

Check the messages in the console window on the IDE to make sure that your code has compiled and verified correctly!

In order to upload your program to the bare bones chip using the BBP, you must open the “Sketch” menu on the Arduino IDE and select “Upload Using Programmer”, see figure 12, above. *NOTE: You MUST use “Sketch > Upload Using programmer” and not click on the right arrow on the IDE toolbar!*

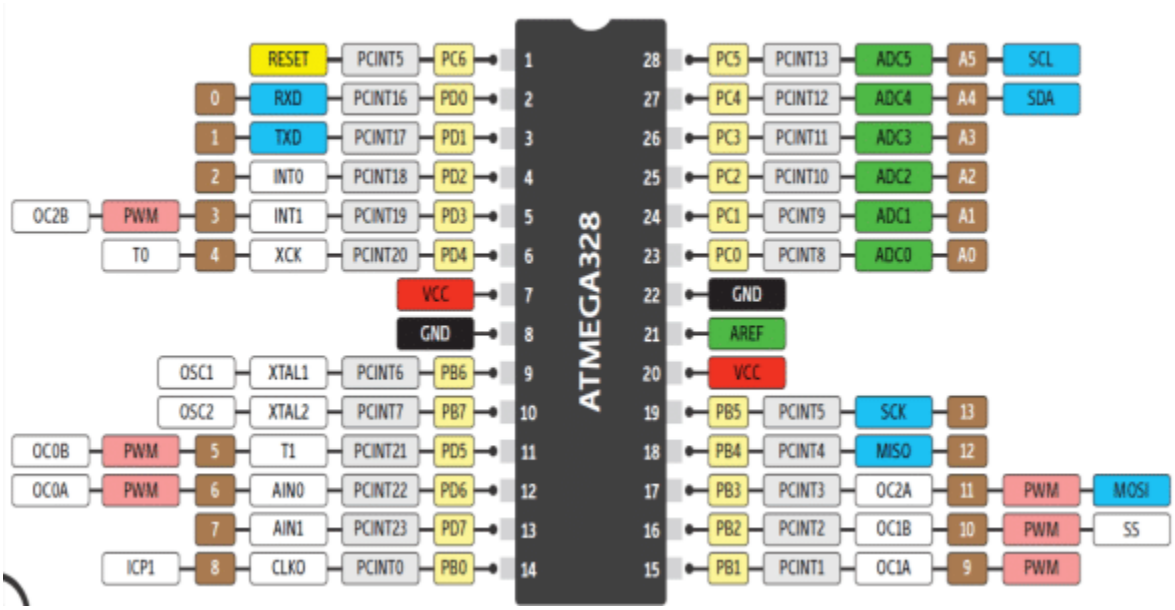
You will see status information (in white and red text) in the IDE console window as this process proceeds. The process will automatically:

- Compile your program (again).
- Upload the compiled program to your “bare bones” microcontroller chip.
- Verify that the program has been correctly stored in the chip’s program memory.

At this point the ATtiny85 will be executing code while in the programmer. If you have flashed a blink program (for an LED in Arduino digital pin 4) to the chip, the red LED by the chip will blink at the correct rate, verifying that everything has been done correctly. You can now create/load the program for your project and select “Sketch > Upload Using Programmer” again to compile and upload the new program.

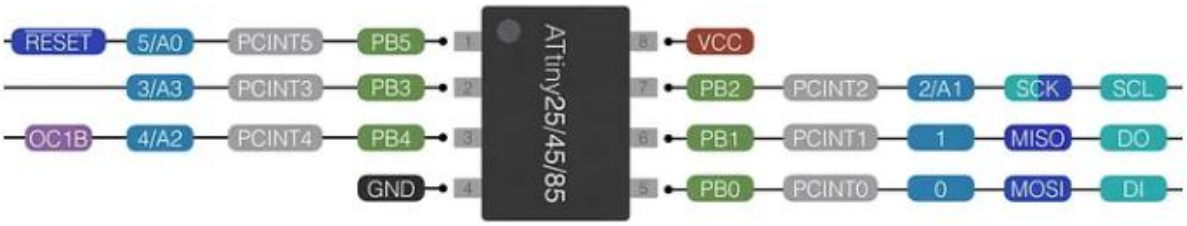
© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to “Terms of Use, License and Disclosure”*

APPENDIX A. ATMEGA328(P) PIN MAPPING.



© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*

APPENDIX B. ATTINY85 PIN MAPPING.



© 2026, Team Practical Projects, Jim Schrempp and Bob Glicksman; all rights reserved. *Subject to "Terms of Use, License and Disclosure"*